PES UNIVERSITY ONLINE

OPERATING SYSTEMS

Process Coordination & Deadlocks

S.Thenmozhi
Department of Computer Applications

Background

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

Background

Suppose that we wanted to provide a solution to the producer-consumer problem that fills ***all*** the buffers.

We can do so by having an integer **`counter`** that keeps track of the number of full buffers. Initially, **`counter`** is set to 0.

It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

Background

```
while (true) {
        /* produce an item in next_produced */
        while (counter == BUFFER_SIZE) ;
                /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

```
while (true) {
        while (counter == 0) ;
                /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next_consumed */
}
```

Background

- **counter++**

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **counter--**

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute	`register1 = counter`	{register1 = 5}
S1: producer execute	`register1 = register1 + 1`	{register1 = 6}
S2: consumer execute	`register2 = counter`	{register2 = 5}
S3: consumer execute	`register2 = register2 - 1`	{register2 = 4}
S4: producer execute	`counter = register1`	{counter = **6**}
S5: consumer execute	`counter = register2`	{counter = **4**}

- **Race condition** – a situation when several processes access the same data concurrently, the outcome of the execution depends on the order of the accesses

Race Condition Illustration

```
static void printchar(char *);
int main()
{
int pid;
if ((pid = fork()) < 0)  {
  printf("fork error");
   }
else if (pid == 0) {
  printchar(“Output from child\n");
}
else {
  printchar(“Output from parent\n");
}
return 0;
}
```

Race Condition Illustration

```
static void printchar(char *str)
{
char *ptr;
int c;
setbuf(stdout, NULL); /* set unbuffered std output*/
for (ptr = str; (c = *ptr++) != 0; )
putc(c, stdout);
}
```

Output

Output froOutpm chiut from ld parent

THANK YOU

S. Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393